AI Compiler Performance Engineering

Auto-Loops Books — ch04

Contents

1 CUDA Hopper/Blackwell: Hardware Properties and Optimization Bounds **1**

1.1 Why Hopper Features Do Not Automatically Speed Up Decode . . 1
1.2 Hopper Memory Hierarchy: Fusion Hard Boundaries 4
1.3 Tensor Memory Accelerator: Hide Latency, Not Traffic 8
1.4 Thread Block Clusters and Distributed Shared Memory 11
1.5 Synchronization: Narrowest Scope Wins 13
1.6 Tensor Core and PTX: Optimize Last 14
1.7 PTX and Softmax Micro-ops 15
1.8 YiRage CUDA Backend: Hardware Constraints as Compile Gates . 16
1.9 Hopper Decode Tuning in Practice 19
1.10 Hopper Benchmarking and Verification 26
1.11 Key Takeaways 29
1.12 Conclusion 30

Bibliography **32**

Index **34**

Chapter 1

CUDA Hopper/Blackwell: Hardware Properties and Optimization Bounds

1.1 Why Hopper Features Do Not Automatically Speed Up Decode

One production team spent a month raising tensor-core utilization on H100—end-to-end batch-1 decode latency fell only ∼3%. Nsight showed excellent TMA overlap and higher TC occupancy, but bytes/token and launches/token were unchanged: activations still exported to HBM between every op boundary (Chen, 2026; Vellaisamy *et al.*, 2026). **Hopper features hide latency and widen on-chip capacity; they do not remove traffic you never fused away.**

Bridge from Chapters 2–3. Chapter 2 said plan residency before exports; Chapter 3 gave the byte budgets (Chen and YiRage Contributors, 2026; Chen, 2020). This chapter instantiates those budgets on Hopper: 228 KB shared memory, TMA, clusters, and tensor cores as *tools in a fixed optimization order*—not a shopping list (NVIDIA, 2022; SemiAnalysis, 2024).

Hopper decode optimization priority (use this order).

1. **Cut activation HBM traffic**—layer fusion (Chapter 2 export-edge mindset); largest gain (Chen, 2026; Dao *et al.*, 2022).

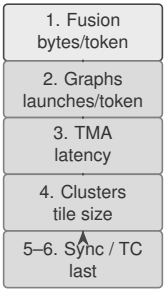


Figure 1.1: Hopper decode optimization priority pyramid: work top-down; do not invert.

Table 1.1: Hopper decode optimization priority and typical batch-1 gains (order matters).

Pri.	Direction	Typical gain	Section
1	Operator fusion (cut activation HBM)	40–80% bytes/token	§1.2
2	CUDA Graph + MegaKernel (cut launches)	30–60% scheduling overhead	§1.9
3	TMA double-buffer (hide latency)	15–30% memory stall	§1.3
4	Thread-block cluster + DSM	10–20% fusion depth	§1.4
5	Minimal sync scope	5–15% pipeline bubbles	§1.5
6	Tensor Core / PTX polish	<10% compute efficiency	§§1.6–1.7

2. **Cut kernel launch overhead**—CUDA Graph + MegaKernel; attacks launches/token (Vellaisamy *et al.*, 2026; NVIDIA, 2024).

3. **Hide memory latency**—TMA double-buffer; does not reduce bytes (SemiAnalysis, 2024).

4. **Widen fusion tiles**—thread-block clusters + DSM (NVIDIA, 2022).

5. **Minimize sync scope**—warp shuffle before block/cluster barriers (Dao *et al.*, 2022).

6. **Tune tensor cores and PTX last**—when bytes/token and launches/token are already clean (Davies *et al.*, 2025; Chen, 2026).

Terminology alignment with Chapters 1–3. We use *kernel launch overhead* for per-token launch tax, *scheduling overhead* for host/framework orchestration, and *data residency* for on-chip lifetime of activation carriers (Vellaisamy *et al.*, 2026; Chen, 2020, 2026). First-use expansions in this chapter: **DSM** (distributed shared memory across a thread-block cluster), **WGMMA** (warpgroup matrix

multiply-accumulate), **SFU** (special function unit for `exp2`), and **FTZ** (flush-to-zero mode for subnormal handling in softmax PTX) (NVIDIA, 2022; Liu *et al.*, 2024; Dao *et al.*, 2022).

Composite lesson. TaxBreak reports hundreds of launches per token on unfused stacks; Memory-Floor shows batch-1 decode often bandwidth- and orchestration-bound before tensor cores saturate (Vellaisamy *et al.*, 2026; Chen, 2026). Profile launches/token and bytes/token first—the same discipline as Chapters 1–3 (Davies *et al.*, 2025).

What you will be able to do. After this chapter you can (1) place Hopper features on the priority pyramid, (2) write a shared-memory line-item budget for a fused layer, (3) decide TMA versus manual loads with profiler evidence, (4) attach ablation rows that separate graph wins from fusion wins, and (5) map YiRage invariants to review checklists (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026). PTX and WGMMA micro-tuning are explicitly last—not because they are unimportant, but because they rarely move batch-1 decode when bytes/token is dirty (Davies *et al.*, 2025; Dao *et al.*, 2022).

Reader prerequisites. Carry Chapter 1 worksheets (launches/token, bytes/token, R_{floor}) and Chapter 3 H100 capacity rows (Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026). When we reference online softmax, assume Chapter 10 numerics; here we only cover *where* carriers live on silicon (Dao *et al.*, 2022, 2023).

Industrial narrative. DeepSeek-scale training stories emphasize co-design under export limits (Liu *et al.*, 2024; Davies *et al.*, 2025). Hopper decode repeats the lesson at SM scale: skillful residency beats brute-force TFLOPs marketing (Vellaisamy *et al.*, 2026; Chen, 2026; Dao *et al.*, 2022). Goodput metrics make the case quantitatively—not peak GEMM banners (Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

How to read this chapter. Skim Figure 1.1 first (Davies *et al.*, 2025). Sections 1.2–1.5 are the tool layer; Section 1.9 is the workflow; Section 1.10 is verification only (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Anti-pattern preview. The five concentrated mistakes at the end of Section 1.9 map directly to inverted pyramid levels—use them in design review before merge (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026). Culture: post the pyramid in your compiler team's Hopper doc template (Chen and YiRage Contributors, 2026).

Cross-chapter dependency map. Chapter 3 listed abstract GPU constraints; this chapter names Hopper mechanisms; Chapters 8–11 specialize TMA, sync, softmax, and MegaKernel codegen (SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026). Skipping here produces CUDA that ignores residency bounds (Vellaisamy *et al.*, 2026; Chen, 2026).

Failure story catalog (short). Shared-memory overflow at max ctx; TMA misalignment on paged KV; illegal cluster on SKU; PTX exp2 drift at long T; graph capture invalidated by driver upgrade (Chen and YiRage Contributors, 2026; NVIDIA, 2024; Dao *et al.*, 2023; Davies *et al.*, 2025). Map symptoms to pyramid level (Table 1.1) before tuning TC tiles (Chen, 2026; Vellaisamy *et al.*, 2026).

TaxBreak anchor. Dense Llama-3.2-1B can average hundreds of kernel launches per output token on unfused H100 stacks (Vellaisamy *et al.*, 2026; Chen, 2026). Those cells are why the pyramid starts with fusion, not TC (Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016).

What this chapter is not. Not a CUDA tutorial on square GEMM microbenchmarks; we document optimization *bounds* for batch-1 decode residency on Hopper-class silicon (NVIDIA, 2022; Chen and YiRage Contributors, 2026; Chen, 2020, 2026).

Organization map. Memory (§1.2), TMA (§1.3), clusters (§1.4), sync (§1.5), TC/PTX (§§1.6–1.7), YiRage (§1.8), practice (§1.9), verify (§1.10) (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

1.2 Hopper Memory Hierarchy: Fusion Hard Boundaries

Decode MegaKernels hold QKV tiles, online softmax carriers, and SwiGLU fragments simultaneously—each must map to a CUDA tier for the kernel lifetime (Dao

Table 1.2: CUDA memory spaces for decode megakernels (Hopper-class).

Space	Scope	Decode role
Register	Thread	Online softmax (m, ℓ, o), MMA fragments
Shared memory	CTA	Fused tiles, TMA staging
Global (HBM)	Grid	Weights, KV cache, spill

et al., 2022; Chen and YiRage Contributors, 2026). Spill to HBM is a bytes/token bill; treat it as a fusion failure (Chen, 2026).

Terminology. A **CTA** (cooperative thread array) is a CUDA thread block—the launch grain that shares one shared-memory pool (NVIDIA, 2022). Hopper exposes up to 228 KB configurable shared memory per SM (227 KB usable per block after reserve) and a 65,536-register file (NVIDIA, 2022).

Shared memory capacity (constraint loop). Engineering meaning: hard ceiling for fused decode tiles—Chapter 3's GPU row, now Hopper-specific (Chen and YiRage Contributors, 2026). **Common pitfall:** tensor bytes only, ignoring padding, double buffers, softmax score tiles (Dao *et al.*, 2022). **Debug:** Nsight Compute shared-memory config; elevated `local memory traffic` signals spill (Davies *et al.*, 2025).

Register pressure. Fusion deepens live ranges; occupancy may drop yet bytes/token still wins when HBM exports disappear (Chen, 2026; SemiAnalysis, 2024). **Debug:** registers per thread and spill counters before MMA tile sweeps (Chen and YiRage Contributors, 2026).

KV versus activation traffic. Weights and KV legitimately stream from HBM each token—fusion targets *activation* exports between attention, softmax, and MLP (Kwon *et al.*, 2023; Chen, 2026; Dao *et al.*, 2022). A common review mistake is celebrating lower DRAM traffic while KV alone explains the counter—diff activation bytes specifically (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Roofline at batch one. Decode GEMMs present thin M with large K/N; arithmetic intensity stays on the memory roof for realistic LLM widths (Goodfellow *et al.*, 2016; Chen, 2026). Registers and shared memory buy effective intensity

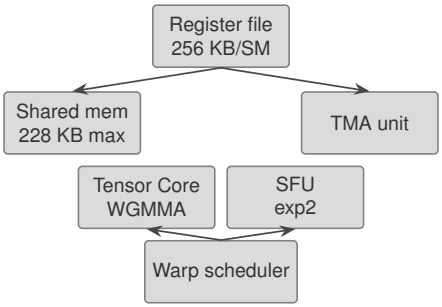


Figure 1.2: Hopper SM resource sketch for decode planning.

without changing FLOP counts—that is the mechanical-sympathy lever Hopper provides (NVIDIA, 2022; SemiAnalysis, 2024).

Blackwell note. Successor SKUs extend TMA and low-precision tensor paths; residency invariants in this section survive—only capacity rows and legality tables change in the chip model (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Cross-hardware note. GPU shared memory is software-managed—flexible but easy to overflow (NVIDIA, 2022). XDNA tile SRAM is fixed at compile time—inflexible but deterministic (Chapter 5) (AMD, 2024b,a). Same Chapter 2 residency goal; different legality mechanics (Chen, 2020).

✓ **Practice checkpoint.** Statically budget shared memory + registers for the full fused region; reject merges that exceed 227 KB usable before codegen (Chen and YiRage Contributors, 2026; Chen, 2026).

Worked tier assignment exercise. For one transformer layer, list every $\Theta(d)$ tensor between attention and MLP: post-norm hidden, Q/K/V fragments, attention output, MLP intermediates, residuals (Dao *et al.*, 2022; Chen, 2026). Mark each as register, shared, or global for your fused plan (Chen and YiRage Contributors, 2026; Chen, 2020). Any global mark between fused stages is a bytes/token bug unless proven dead (Vellaisamy *et al.*, 2026; Chen, 2026).

Spill versus intentional HBM. Intentional traffic: weights, KV appends, final layer outputs to framework buffers (Kwon *et al.*, 2023; Goodfellow *et al.*,

2016). Spill traffic: softmax carriers, score tiles, or MLP fragments written because shared memory overflowed (Chen, 2026; Dao *et al.*, 2022; Davies *et al.*, 2025). Nsight distinguishes them via local memory loads and stack-like access patterns (SemiAnalysis, 2024).

Occupancy versus fusion trade. High shared-memory configs lower resident blocks per SM yet may still win on ms/token when fusion deletes HBM passes (Chen, 2026; SemiAnalysis, 2024; Vellaisamy *et al.*, 2026). Optimize occupancy only after proving legality and bytes/token improvement (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Lab charts that show "low occupancy" without bytes/token context mislead reviewers (Chen, 2026).

Example unfused traffic. Llama-class stacks often materialize six to eight $\Theta(d)$ activation buffers per layer at batch-1 (Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Chen, 2026). At d=4096 BF16, six round trips imply roughly $6 \times 4096 \times 2 \approx 49\,\text{KB}$ activation traffic per layer per token before KV reads (Kwon *et al.*, 2023; Chen, 2026). Fusion targets that bill first (Chen, 2020; Chen and YiRage Contributors, 2026).

Shared-memory bank conflicts (constraint loop). Engineering meaning: legal capacity can still serialize when strided layouts hit the same 32-way bank set—Chapter 3's GPU row, now layout-specific (NVIDIA, 2022; Shah *et al.*, 2024). **Common pitfall:** padding score tiles for capacity but leaving WGMMA-unfriendly strides (Dao *et al.*, 2022). **Debug:** Nsight shared-memory conflict counters; compare swizzled versus naive layouts before blaming TMA (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Hopper fusion must swizzle for **WGMMA** (warpgroup matrix multiply-accumulate) consumption, not only fit byte totals (SemiAnalysis, 2024; Shah *et al.*, 2024).

Chapter 2 iron rule link. Iron Rule 2 (keep activations on-chip across stages) is implemented here as shared-memory and register budgets (Chen, 2020, 2026; Dao *et al.*, 2022). Violating the rule exports to global—bytes/token rises even if TMA overlap looks healthy (SemiAnalysis, 2024; Vellaisamy *et al.*, 2026).

L2 hidden export. Even legal shared-memory plans can lose if every stage writes full vectors through L2 to HBM (Chen, 2026; Kwon *et al.*, 2023). Fusion must minimize round trips, not only fit in shared memory while exporting intermediates (Chen, 2020; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026).

Multi-hardware delta (again). CPU L3 blocking and XDNA static tiles solve the same export problem with different legality (AMD, 2024b; Goodfellow *et al.*, 2016; Rotem *et al.*, 2018; Davies *et al.*, 2025). Hopper's programmable shared memory is powerful and easy to overflow—that is the engineering tension (NVIDIA, 2022; Chen and YiRage Contributors, 2026; Chen, 2026).

Memory tier cheat sheet for reviewers. When reviewing CUDA diffs ask: where do QKV fragments live between ops? Where do (m, ℓ, o) live in the KV loop? Where does SwiGLU intermediate sit? (Dao *et al.*, 2022; Chen, 2026; Chen and YiRage Contributors, 2026) Any answer of "global" between fused stages is a regression unless proven dead (Chen, 2020; Vellaisamy *et al.*, 2026).

1.3 Tensor Memory Accelerator: Hide Latency, Not Traffic

TMA (Tensor Memory Accelerator) issues asynchronous bulk copies from global memory to shared memory while MMA units compute on the previous tile (NVIDIA, 2022; SemiAnalysis, 2024). TMA overlaps latency; only fusion reduces activation bytes/token (Chen, 2026).

Applicable when. Tile boxes 128×128 with contiguous global addresses and amortized descriptor setup—weight blocks, contiguous KV slices, steady double-buffer pipelines inside a MegaKernel (SemiAnalysis, 2024; Dao *et al.*, 2023).

Avoid when. Batch-1 M=1 micro-tiles, irregular paged-KV gathers, or descriptor setup dominating copy time (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Profile TMA versus vector loads on the target shape (Chen and YiRage Contributors, 2026).

Common pitfalls. Double-buffer handoff bugs (compute on buffer being overwritten); paged KV non-contiguous addresses; mandating TMA on thin tiles (Dao *et al.*, 2023; Davies *et al.*, 2025).

Debug. Nsight Compute: `TMA busy cycles` vs `MMA busy cycles`—ratio below ∼0.3 suggests weak TMA benefit; check `local memory loads` for spill (SemiAnalysis, 2024).

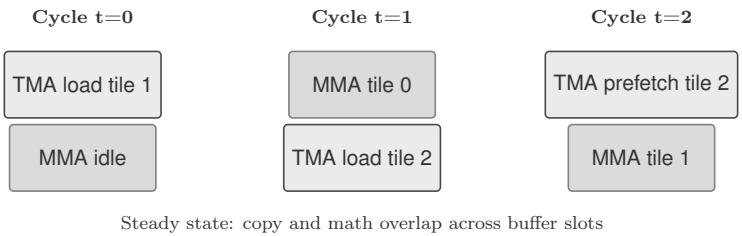


Figure 1.3: TMA double-buffer timeline: async copy overlaps tensor-core math.

Paged KV co-design. vLLM block tables map logical tokens to non-contiguous physical blocks (Kwon *et al.*, 2023). TMA descriptors assume regular layouts—MegaKernels rebuild or index descriptors per block inside the KV loop (Dao *et al.*, 2023; Chen and YiRage Contributors, 2026). This is production-hard; Chapter 8 returns to double-buffer theory (SemiAnalysis, 2024).

Manual load fallback. Vectorized global loads remain valid when TMA setup dominates—YiRage should lower both paths and select with a profile-guided cost model, not a global compile flag (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Teams that mandate TMA in code review without shapes invite the anti-pattern in Section 1.9 (Davies *et al.*, 2025).

Batch-1 thin-M reality. Many decode GEMMs present M=1 with large inner dimension (Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026). TMA shines when copy boxes amortize over deep pipelines inside a MegaKernel—not when each score fragment is a separate descriptor (SemiAnalysis, 2024; Chen and YiRage Contributors, 2026). Profile on *your* head dimension and KV block size, not a square GEMM microbench (Davies *et al.*, 2025; Chen, 2026).

Energy angle. HBM transactions dominate decode energy (Chen, 2026; Davies *et al.*, 2025). Fusion bytes/token savings matter for TCO even when ms/token gains look modest in lab cells (Vellaisamy *et al.*, 2026; AMD, 2024b). TMA overlap saves time; fusion saves joules per token (SemiAnalysis, 2024; Chen, 2026).

Synchronization case study. Double-buffered TMA needs one barrier after copy completes and one before overwriting the consumed buffer (SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; NVIDIA, 2022). Extra barriers after each warp shuffle in softmax add pipeline bubbles without safety benefit (Dao *et al.*,

2022; Goodfellow *et al.*, 2016). Compiler sync insertion should be edge-driven on the fused DAG (Chen, 2020; Chen and YiRage Contributors, 2026).

Long-context stability. TMA misalignment bugs may hide at T=512 and explode at T=8192 (Dao *et al.*, 2023; Davies *et al.*, 2025). Always include long-ctx buckets in Hopper validation—not only the bucket your laptop tests (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

Observability hooks. Stable kernel names derived from pass hashes help profilers map samples to bufferization tiers (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025). Production builds may strip debug tags but should retain residency class notes in release artifacts (Vellaisamy *et al.*, 2026; Chen, 2026).

Warp-group context. Hopper issues WGMMA from warpgroups of four warps—thirty-two threads each (Shah *et al.*, 2024; NVIDIA, 2022). Fusion planners must align head partitions and tile shapes to warpgroup granularity or leave MMA lanes idle (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

First-token versus steady state. Compile and capture costs hit first token; steady decode steps reveal residency wins (NVIDIA, 2024; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Benchmark both when product SLOs include cold start (Davies *et al.*, 2025; Chen, 2026).

Descriptor lifecycle. Host or device code builds a TMA descriptor with global layout, shared destination, and copy box (NVIDIA, 2022; SemiAnalysis, 2024). Weight descriptors are often static per compiled bucket; KV descriptors may change with block-table indices each iteration (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026). Misaligned layouts produce silent numerical drift—always pair kernel changes with FP64 reference diffs at long ctx (Dao *et al.*, 2023).

Figure 1.3 usage. The timeline is the steady-state target—getting there requires correct double-buffer handoffs and legal tile sizes (SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Dao *et al.*, 2022). If steady state never appears in timelines, fix fusion legality before tuning descriptor fields (Davies *et al.*, 2025; Chen, 2026).

Chapter 2 link. TMA implements *pipeline movement* on-chip—moving data while math runs—but does not replace export-edge removal (Chen, 2020, 2026).

Cluster versus single-block decision tree. If fused tile fits in 227 KB usable with margin, stay single-block (NVIDIA, 2022; Chen and YiRage Contributors, 2026). If score tile or SwiGLU buffer exceeds block cap but fits pooled DSM, plan cluster (SemiAnalysis, 2024; Dao *et al.*, 2023). If neither fits, split fusion at Chapter 2 export edge—do not spill (Chen, 2026, 2020; Vellaisamy *et al.*, 2026).

Flash-Decoding without extra launches. KV splits across SMs need in-kernel online softmax merges (Dao *et al.*, 2023, 2022). Extra merge kernels reopen launches/token—design cluster or in-kernel merge before accepting multi-kernel fallback (Vellaisamy *et al.*, 2026; NVIDIA, 2024; Chen and YiRage Contributors, 2026).

✓ **Practice checkpoint.** Confirm double-buffer invariants in code review; one barrier after TMA complete, one before buffer reuse (Chen and YiRage Contributors, 2026; SemiAnalysis, 2024).

1.4 Thread Block Clusters and Distributed Shared Memory

Hopper **thread-block clusters** pool **DSM** (distributed shared memory) across cooperating CTAs—widening the on-chip fusion envelope without an HBM export (NVIDIA, 2022; SemiAnalysis, 2024).

Applicable when. Score tiles or fused buffers exceed single-block shared memory; Flash-Decoding-style KV splits need cross-block softmax reductions (Dao *et al.*, 2023, 2022).

Avoid when. Footprint already fits one block—clusters add sync and launch complexity without residency win (Chen and YiRage Contributors, 2026).

Common pitfalls. Cluster launch illegal on target SKU; DSM double-counted in planners; fallback merge kernel reintroduces launches/token (Vellaisamy *et al.*, 2026; Chen, 2020).

Table 1.3: Cluster DSM vs. single-block shared memory (decode).

Aspect	**Single CTA**	**Cluster + DSM**
Peak scratch	One block SMEM	Pooled DSM
Softmax max-reduce	Warp/block	Cross-CTA on-chip
XDNA analogue	One tile SRAM	Multi-tile DMA graph

Debug. Compile-time cluster legality report; Nsight timeline for extra barriers; compare launches/token with single-block plan (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

✓ **Practice checkpoint.** YiRage must reject illegal cluster dims at compile time—no silent multi-kernel fallback (Chen and YiRage Contributors, 2026; Chen, 2020).

Flash-Decoding link. When KV is split across SMs, partial softmax statistics must merge on-chip (Dao *et al.*, 2023, 2022). An extra merge kernel reopens launches/token—cluster DSM exists to keep the merge inside the MegaKernel capture region (Vellaisamy *et al.*, 2026; NVIDIA, 2024). If your SKU forbids the cluster shape, document the multi-kernel fallback cost explicitly in the worksheet (Davies *et al.*, 2025).

Capacity accounting with DSM. Cluster pooling increases peak scratch but also increases barrier scope (NVIDIA, 2022; SemiAnalysis, 2024). Planners must sum DSM bytes across CTAs in the cluster—double-booking is as fatal as single-block overflow (Chen and YiRage Contributors, 2026; Chen, 2026). Chapter 3's inequality mindset applies; only the geometry changes (Chen, 2020).

Comparison to CPU AVX paths. CPU decode uses cache-blocked GEMMs and vectorized exp2 on softmax (Goodfellow *et al.*, 2016; Davies *et al.*, 2025; AMD, 2024b). Hopper wins when fusion keeps data on SM; CPU wins when GPU launch storms dominate at low QPS (Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026). Triplet tests prevent declaring victory on GPU ms/token alone (AMD, 2024a; Davies *et al.*, 2025).

Grid sync warning. Device-wide synchronization stalls every CTA and destroys graph-capture friendliness (Vellaisamy *et al.*, 2026; NVIDIA, 2024, 2022). If an

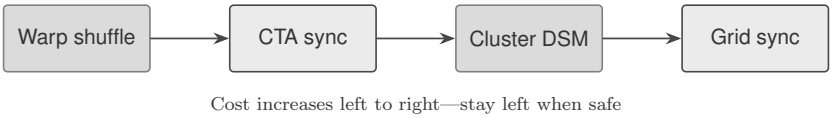


Figure 1.4: CUDA synchronization hierarchy for decoder megakernels.

algorithm seems to need grid sync per token, revisit fusion boundaries or cluster merges first (Dao *et al.*, 2023; Chen and YiRage Contributors, 2026).

Producer-consumer sync edges. List edges in the fused DAG where shared-memory buffers hand off between TMA, MMA, and epilogue phases (Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024). Insert barriers only on those edges—extra block syncs show as bubbles in Nsight timelines (Davies *et al.*, 2025; Dao *et al.*, 2022).

Figure 1.4 usage. Pick the leftmost mechanism that preserves correctness—grid sync is almost never the answer in per-token decode (Vellaisamy *et al.*, 2026; NVIDIA, 2022; Dao *et al.*, 2022). Chapter 9 expands sync insertion algorithms (Chen and YiRage Contributors, 2026; Chen, 2020).

1.5 Synchronization: Narrowest Scope Wins

Unnecessary barriers serialize TMA pipelines (SemiAnalysis, 2024). Place sync only on shared-memory phase edges in the fused dataflow graph (Chen and YiRage Contributors, 2026; Chen, 2020).

Hierarchy (cheapest first). **Warp shuffle** for row-wise softmax max/sum (Dao *et al.*, 2022). **CTA block sync** for TMA/MMA buffer handoffs (SemiAnalysis, 2024). **Cluster barrier** for DSM reductions (NVIDIA, 2022). **Grid sync**—avoid in per-token decode loops (Vellaisamy *et al.*, 2026).

✓ **Practice checkpoint.** Audit fused kernels for barriers after every shuffle—remove bubbles without safety benefit (Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

CUDA Graphs interaction. Graph capture removes host launch overhead but does not shrink shared-memory footprints (NVIDIA, 2024; Chen, 2026). Capture

Table 1.4: Decode diagnostics on Hopper (what to fix first).

Signal	Limiter	First action
High launches/token	Scheduling overhead	Graph / MegaKernel
High bytes/token	HBM residency	Fusion
High TC util, flat ms/token	Wrong metric	Return to bytes/token

after fusion is legal; capture instead of fusion is not (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Systems timelines should show fewer host packets *and* lower DRAM bytes per token when both levers apply (Davies *et al.*, 2025).

Warp specialization preview. Persistent decode kernels assign warps to TMA issue, MMA compute, and epilogue roles (SemiAnalysis, 2024; NVIDIA, 2022; Vellaisamy *et al.*, 2026). Chapter 11 MegaKernels assume this model (Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026). Sync placement must respect role boundaries—a barrier in the wrong warp group stalls the whole pipeline (Davies *et al.*, 2025).

1.6 Tensor Core and PTX: Optimize Last

At batch-1, MMA tiles are thin—tensor cores often sit idle while DRAM and launches dominate (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). **Do not open with TC tile sweeps** when R_{floor} is low (Chapter 1) (Chen, 2026).

Quantitative intuition. Removing six d-vector HBM round trips at d=4096 BF16 saves ∼49 KB activation traffic per layer per token—often larger than a 10-point TC utilization gain on 1×4096 GEMM (Chen, 2026; Vellaisamy *et al.*, 2026). After fusion cleans bytes/token, TMA-fed MMA epilogues matter (SemiAnalysis, 2024; Dao *et al.*, 2022).

SFU versus MMA balance. Attention decode interleaves WGMMA matmuls with SFU-heavy softmax (Dao *et al.*, 2022; NVIDIA, 2022). Profiling both pipelines prevents false "TC-bound" diagnoses when softmax is the long pole inside the same SM (Davies *et al.*, 2025; Liu *et al.*, 2024). Refer back to Section 1.6 before opening tile configurators (Chen, 2026).

Quantization on Hopper. FP8 and INT8 weights need dequant in-register or in shared memory (Liu *et al.*, 2024; Davies *et al.*, 2025; NVIDIA, 2022). Scales consume the same residency budget as softmax carriers—co-decide with fusion grouping (Chen, 2026; Chen and YiRage Contributors, 2026). Dropping precision without retile can change padding and spill behavior (Chen, 2020).

Blackwell FP4 trajectory. Successor chips advertise wider low-precision tensor paths (Liu *et al.*, 2024; NVIDIA, 2022; Davies *et al.*, 2025). Porting re-tiles kernels but preserves the pyramid: bytes, launches, latency, TC (Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024).

Team A versus Team B (retold). Team A improves MMA tile from 1% to 3% TC util; Team B fuses an MLP boundary saving ∼40% activation bytes—only Team B moves p99 at batch-1 when R_{floor} is low (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022). Use this story in backlog prioritization (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Table 1.4 usage. Match profiler signals to the "first action" column before opening code (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026). High TC util with flat ms/token is a diagnostic row, not a success row (Chen, 2026; Dao *et al.*, 2022).

PTX review culture. Softmax-related PTX changes are not "micro-optimizations"—they are ABI changes (Dao *et al.*, 2022; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Treat them with the same rigor as fusion grouping changes (Chen, 2020; Dao *et al.*, 2023).

1.7 PTX and Softmax Micro-ops

Online softmax uses **SFU** (special-function unit) pipelines for **exp2**-style exponentials (Dao *et al.*, 2022; NVIDIA, 2022). **WGMMA** (warp-group matrix multiply-accumulate) is Hopper's tensor-core instruction issued by warpgroups of four warps (Shah *et al.*, 2024).

PTX as ABI. Reviewed `ex2.approx` sequences with explicit max-subtraction and documented **FTZ** (flush-to-zero) behavior (Liu *et al.*, 2024; Davies *et al.*, 2025).

Changing PTX without FP64 reference diffs at T=8192 breaks fleet determinism (Dao *et al.*, 2023; Chen and YiRage Contributors, 2026).

Advance notice. This subsection needs basic CUDA/PTX literacy; skipping it does not block fusion or TMA work above (Goodfellow *et al.*, 2016).

✓ **Practice checkpoint.** Softmax PTX changes require FP64 diffs at ctx $\in \{512, 2048, 8192\}$ before merge (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026).

Inline PTX in production. DeepSeek-style kernels select SFU instructions predictably rather than hoping the compiler discovers them (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026). Separate softmax launches break MegaKernel boundaries and reopen launches/token (Vellaisamy *et al.*, 2026; Chen, 2026; Dao *et al.*, 2022). PTX belongs inside the fused region, not as a follow-on kernel (Chen and YiRage Contributors, 2026; Chen, 2020).

1.8 YiRage CUDA Backend: Hardware Constraints as Compile Gates

Advance notice. This section assumes compiler and IR literacy; backend engineers should read it end-to-end, while kernel tuners can skim invariants and return for lowering detail (Chen, 2020; Chen and YiRage Contributors, 2026).

YiRage lowers the same decode IR to CUDA, CPU, and XDNA—Hopper legality is dynamic residency checking against chip models from Chapter 3 (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b).

Five compiler invariants (decode mode).

1. **No activation-sized global stores** inside a fused region—*YiRage:* `BufferizationPass` rejects illegal stores at compile time instead of emitting silent spill (Chen, 2026; Chen and YiRage Contributors, 2026).
2. **TMA legality is per-copy**—*YiRage:* `TmaSelectionPass` chooses TMA or vector loads per copy edge (SemiAnalysis, 2024; Vellaisamy *et al.*, 2026).

Table 1.5: YiRage Hopper lowering: IR in, check out.

Pass	**Input → action**	**Output check**
Bufferization	Linalg/tensor IR → tier assignment vs. chip model	No illegal global activations
Fusion grouping	Bufferized graph → export-edge merges	Shared/reg high-water ≤ caps
TMA selection	Copy edges → descriptor or vector path	Per-edge legality report
Cluster planning	Oversized tiles → DSM cluster shape	Compile fail if illegal
Sync insertion	Fused DAG → minimal barriers	No grid sync per token
PTX lowering	Softmax region → SFU `exp2` ABI	FP64 diff at long ctx

3. **Cluster dims are compile-time contracts**—*YiRage:* `ClusterPlanningPass` fails the build on illegal DSM geometry (NVIDIA, 2022; Chen and YiRage Contributors, 2026).

4. **Softmax PTX is versioned ABI**—*YiRage:* `PtxLoweringPass` pins reviewed `exp2` sequences; CI diffs long-ctx numerics (Dao *et al.*, 2022; Liu *et al.*, 2024).

5. **Triplet bufferization**—*YiRage:* `FusionGroupingPass` must export legality summaries for CUDA, CPU, and XDNA backends (AMD, 2024a; Davies *et al.*, 2025).

Invariant enforcement in CI. Decode-mode builds treat activation-sized global stores as hard errors; TMA and cluster passes emit legality reports consumed by nightly triplet jobs (Chen and YiRage Contributors, 2026; AMD, 2024b; Chen, 2020). Hand-tuned CUDA that bypasses the model rots on the next SKU stepping—keep capacities in the Chapter 3 chip model (Davies *et al.*, 2025; IREE / iree-org contributors, 2024).

Handoff preview. Chapter 5 shows the same IR on XDNA: static tile slots replace dynamic shared memory (AMD, 2024b,a). Triplet parity is not optional

for a multi-hardware compiler narrative (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Documentation deliverables per promotion. Residency diagram, TMA descriptor notes, cluster dims, sync plan, PTX softmax changelog, TaxBreak launches/token cell, Memory-Floor bytes/token cell (Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Partners cannot replay regressions when benchmarks live only in private spreadsheets (Chen, 2020).

Testing gates. Numerical diffs at long ctx; Nsight DRAM delta at batch-1; triplet compile; graph bucket hit rate across three ctx lengths (Dao *et al.*, 2023; NVIDIA, 2024; AMD, 2024b; Chen and YiRage Contributors, 2026).

Lowering walkthrough detail. Each row in Table 1.5 must emit artifacts CI can diff—legality reports are not optional debug prints (Chen and YiRage Contributors, 2026; Chen, 2020).

CUDA Graphs and bufferization. Graph capture pins loop bounds and launch structure (NVIDIA, 2024; Kwon *et al.*, 2023). Bufferization must allocate for max ctx in the captured bucket—growing block tables without re-capture is an open product tension (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Liu *et al.*, 2024).

IREE-style portability. Modular retargeting treats backends as plugins over shared IR (IREE / iree-org contributors, 2024; Chen and YiRage Contributors, 2026; Chen, 2020). Hopper capacities live in the plugin model, not scattered in hand-written kernels (Davies *et al.*, 2025; NVIDIA, 2022).

Silent spill signature. Flat GPU utilization with rising HBM throughput often means spill or export edges (Chen, 2026; NVIDIA, 2022; Davies *et al.*, 2025). Map telemetry to pyramid level before opening TC configurators (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

✓ **Practice checkpoint.** Hopper PRs attach residency diagram + legality report; triplet export compiles or documents deferral (Chen and YiRage Contributors, 2026; AMD, 2024b).

1.9 Hopper Decode Tuning in Practice

This section consolidates profiling workflow, reusable patterns, a quantified case study, and concentrated anti-patterns—the material formerly scattered across profiling, patterns, and compiler-reference sections (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

1.9.1 Profile-first workflow

Start from median ms/token at BS=1; decompose launches/token (TaxBreak) and bytes/token (Memory-Floor / Nsight DRAM) (Vellaisamy *et al.*, 2026; Chen, 2026). Nsight Systems for launch storms and graph replay; Nsight Compute for spill, DRAM bytes, TMA vs MMA busy cycles (Davies *et al.*, 2025; SemiAnalysis, 2024). **Order:** end-to-end token latency → bytes → launches → TC utilization (Chen, 2026).

Systems versus Compute. Systems answers "how many launches and how much host noise"; Compute answers "did we spill and did DRAM bytes drop" (Davies *et al.*, 2025; SemiAnalysis, 2024; NVIDIA, 2024). A fused kernel can lose isolated kernel timers yet win end-to-end ms/token when launch storms disappear (Vellaisamy *et al.*, 2026; Chen, 2026). Never promote based on kernel-only timers alone (Davies *et al.*, 2025).

Register carriers for online softmax. Each head keeps (m, ℓ) scalars and a partial output vector—often FP32 even when matmul uses BF16 (Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Sixteen heads multiply register pressure; warp-specialized layouts assign warpgroups per head cluster to keep shuffle reductions in-register (NVIDIA, 2022; Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). This is why the shared-memory sheet and register sheet are one joint budget (Chen, 2026).

Postmortem sequence. When Hopper decode regresses after a merge: diff launches/token, replay bytes/token worksheet, compare DRAM delta on fused kernel, bisect graph vs fusion, *then* consider driver or clock issues (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; NVIDIA, 2024). Skipping bytes/token replay wastes days on launch-only noise (Chen and YiRage Contributors, 2026).

Teaching versus production CUDA. Courses emphasize square GEMM microbenchmarks; production decode emphasizes residency graphs and goodput counters (Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026). This chapter bridges that gap for Hopper specifically (SemiAnalysis, 2024; NVIDIA, 2022).

Roadmap to later chapters. Chapter 8 specializes TMA double-buffer theory; Chapter 9 sync hierarchy; Chapter 10 softmax carriers; Chapter 11 full MegaKernel assembly (Dao *et al.*, 2022, 2023; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Hopper bounds here are prerequisites—later chapters reference these labels rather than re-derive capacities (NVIDIA, 2022; SemiAnalysis, 2024).

Open research threads. Dynamic shapes without re-capture, multi-GPU tensor-parallel MegaKernels, and MoE expert-local fusion remain active (Liu *et al.*, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025). This chapter documents single-GPU batch-1 baselines engineers can extend (Vellaisamy *et al.*, 2026; Chen, 2026).

Reader exercises. (1) Sketch shared-memory allocation for one fused layer at d=2048. (2) Explain when TMA hurts batch-1 decode. (3) Fill Table 1.6 for your model (Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; SemiAnalysis, 2024).

Industrial narrative (closing practice section). Goodput engineering on Hopper is measurable: launches/token, bytes/token, ms/token together (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Skillful residency beats SKU shopping when R_{floor} is low (Chen, 2026; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026).

Pattern usage in review. Require Pattern 1 skeleton diagram on every layer-fusion PR; Pattern 5 ablation on every graph PR; Pattern 6 on every IR change touching bufferization (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025).

Documentation contract (reprise). Ship residency diagram, TMA notes, cluster dims, sync plan, softmax PTX changelog, and goodput cells with each promotion (Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; SemiAnalysis, 2024).

Energy and TCO (short). Fusion bytes/token savings reduce HBM energy even when ms/token gains look modest (Chen, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; AMD, 2024b). Cloud TCO cares about joules per useful token—not peak TFLOPs (Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Operator documentation. Version benchmark artifacts in artifact stores partners can replay (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026). Private spreadsheets do not scale across fleets (Davies *et al.*, 2025).

Quantization-aware benchmarks (short). INT8 and FP8 paths need explicit scale tensors in exports so fusion legality does not fold away dequant boundaries (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2026). Include per-channel scales typical of long-context decode in nightly cells (Davies *et al.*, 2025; Kwon *et al.*, 2023).

1.9.2 Engineering patterns

Pattern 1: Fused layer skeleton. Scene: single-layer decode on Hopper (Vellaisamy *et al.*, 2026). **Solution:** TMA load → QKV MMA → KV loop with online softmax → SwiGLU → residual—no activation HBM stores (Dao *et al.*, 2022; SemiAnalysis, 2024). **Benefit:** ∼6–8 HBM round trips removed; launches drop from ∼12 to 1 per layer (Chen, 2026). **Notes:** pre-compute shared-memory + register budget (Section 1.2) (Chen and YiRage Contributors, 2026).

Pattern 2: Context buckets + CUDA Graphs. Scene: production ctx $\in \{512, 2048, 8192\}$ (Kwon *et al.*, 2023; NVIDIA, 2024). **Solution:** separate captured graphs per bucket; static loop bounds inside MegaKernel (Chen and YiRage Contributors, 2026). **Benefit:** launches/token collapse without changing math (Vellaisamy *et al.*, 2026). **Notes:** graphs without fusion still pay full bytes/token (Chen, 2026).

Pattern 3: Warp-specialized TMA. Scene: KV loop memory-bound (SemiAnalysis, 2024). **Solution:** dedicated warps issue TMA while MMA warps consume previous tile (NVIDIA, 2022). **Benefit:** raises TMA/MMA overlap ratio (Davies *et al.*, 2025). **Notes:** misbalanced roles stall the pipeline (Chen and YiRage Contributors, 2026).

Table 1.6: Llama-3.2-1B decode layer: operator-first vs Hopper fusion (H100, batch-1, ctx=2048).

Metric	Operator-first	Hopper fusion	Change
Launches/token (layer)	12	1	−91.7%
Activation HBM bytes/token	∼49 KB	∼8 KB	−83.7%
Median ms/token (layer)	42 µs	19 µs	−54.8%
Tensor Core utilization	12%	15%	+25% (minor)

Pattern 4: Softmax on shuffle + SFU. Scene: attention decode (Dao *et al.*, 2022). **Solution:** max via warp shuffle; **exp2** on SFU; carriers in FP32 (Goodfellow *et al.*, 2016; Liu *et al.*, 2024). **Benefit:** avoids extra global softmax materialization (Chen, 2026). **Notes:** Chapter 10 owns numerics; this chapter owns placement (Dao *et al.*, 2022).

Pattern 5: Graph + fusion composition. Scene: H100 cell with low R_{floor} (Chen, 2026). **Solution:** fuse first, capture graph second (NVIDIA, 2024; Vellaisamy *et al.*, 2026). **Benefit:** bytes and launches both move (Davies *et al.*, 2025). **Notes:** ablation rows must label which lever moved (Chen, 2026).

Pattern 7: PagedAttention inside MegaKernel. Scene: production serving with vLLM block tables (Kwon *et al.*, 2023). **Solution:** device-side block index walk; gather or per-block TMA inside KV loop (Dao *et al.*, 2023; Chen and YiRage Contributors, 2026). **Benefit:** keeps paging without per-op launches (Vellaisamy *et al.*, 2026). **Notes:** hardest Hopper integration point—profile at 8K and 32K ctx (Davies *et al.*, 2025).

Pattern 6: Triplet parity. Scene: fleet with GPU + NPU endpoints (AMD, 2024b; Davies *et al.*, 2025). **Solution:** same IR, three backends; compare normalized bytes/token (Chen and YiRage Contributors, 2026). **Benefit:** prevents GPU-only schedule drift (Chen, 2020).

1.9.3 Worked example: Llama-3.2-1B on H100

Setup: batch-1, ctx=2048, hidden d=2048, 16 Q heads / 4 KV heads (GQA) (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).

Table 1.7: Shared-memory budget for Llama-3.2-1B fused layer (H100, d=2048, ctx=2048).

Buffer	**Tier**	**Bytes**	**Notes**
Hidden staging	Shared	8 KB	RMSNorm input tile
QKV double buffer	Shared	48 KB	TMA ping-pong
Softmax scores	Shared	32 KB	One score tile (no duplicate)
SwiGLU tiles	Shared	64 KB	Up/down projections
Epilogue / residual	Shared	16 KB	In-place when aligned
Subtotal		**168 KB**	of 227 KB usable
TMA descriptor scratch	Shared	∼8 KB	Headroom margin

Table 1.8: Ablation: graph capture vs. fusion vs. composition (same H100 cell).

Configuration	**Launches/token**	**Bytes/token**	**Layer ms/token**
Baseline (eager, unfused)	12	∼49 KB	42 µs
Graph capture only	1	∼49 KB	35 µs
Fusion only (no graph)	12	∼8 KB	24 µs
Graph + fusion (shipped)	1	∼8 KB	19 µs

Reading the ablation rows. Row 2 proves graph capture attacks scheduling overhead only—launches/token drops from 12 to 1 while bytes/token stays at ∼49 KB (NVIDIA, 2024; Vellaisamy *et al.*, 2026). Row 3 proves fusion attacks activation traffic—bytes fall to ∼8 KB but launches remain at 12 without capture (Chen, 2026; Chen and YiRage Contributors, 2026). Row 4 is the shipped configuration: both levers move, yielding the full ∼55% layer latency win in Table 1.6 (Davies *et al.*, 2025). Reviewers should reject PRs that cite only one row without labeling which pyramid level moved (Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026). Adding a convenience buffer beyond Table 1.7 triggers spill or occupancy collapse—recompute the sheet before merge (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022).

Profile interpretation for the case. After fusion, Systems should show one launch per layer; Compute should show lower DRAM throughput with similar MMA instruction counts (Vellaisamy *et al.*, 2026; NVIDIA, 2024; Davies *et al.*, 2025). If TC utilization rises but Table 1.6 ms/token is flat, the team optimized the wrong row (Chen, 2026). Contrast with L4 and XDNA exports of the same IR to keep fleet decisions honest (AMD, 2024b; Chen, 2026; Chen and YiRage Contributors, 2026).

Unfused baseline narrative. Operator-first stacks pay TaxBreak launch counts *and* Memory-Floor activation bytes (Vellaisamy *et al.*, 2026; Chen, 2026). Hopper fusion attacks both; graphs alone attack only launches (NVIDIA, 2024; Davies *et al.*, 2025). Table 1.6 is the template for your model row—replace numbers, keep columns (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Review question. A teammate proposes TMA on every 1×128 score fragment but still stores attention output to global before MLP (Chen, 2026; Dao *et al.*, 2022). Ask: what happens to bytes/token and Chapter 10 carrier residency? (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022) Correct answer: bytes stay high; MegaKernel legality fails despite pretty TMA timelines (Chen, 2026; Davies *et al.*, 2025).

KV loop structure. For each KV block: TMA K/V tiles, MMA $QK^\top$, online softmax update, MMA PV accumulate (Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023). The loop body must stay inside the same capture region as QKV and MLP—exporting partial attention between blocks reopens bytes/token (Chen, 2026; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Residual update placement. Final residual can be in-place in shared memory when dimensions align (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). A global residual write mid-layer is another $\Theta(d)$ export—flag in review (Chen, 2026, 2020).

GQA head accounting. Grouped-query attention reduces KV heads versus Q heads—KV loop traffic scales with KV head count, not Q count (Kwon *et al.*, 2023; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Worked example uses 16 Q / 4 KV; adjust line items for your architecture (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Figure 1.5 usage. Use in design reviews to explain why TMA alone cannot fix operator-first graphs (SemiAnalysis, 2024; Chen, 2026; Vellaisamy *et al.*, 2026). The diagram is the export-edge story from Chapter 2 drawn for one layer (Chen, 2020; Dao *et al.*, 2022).

Anti-pattern review cadence. Run the five anti-patterns as a checklist in weekly Hopper office hours—not only at merge time (Davies *et al.*, 2025; Chen,

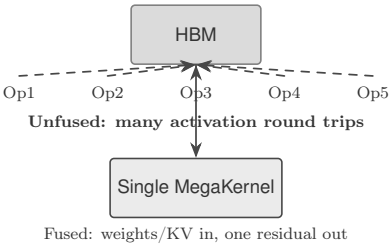


Figure 1.5: Fusion removes activation HBM ping-pong; TMA does not.

2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Teams that only check at release discover spill in production (Chen, 2026; NVIDIA, 2022).

Ablation discipline. Publish graph-only, fusion-only, and composed rows—label which metric moved (Chen, 2026; NVIDIA, 2024; Vellaisamy *et al.*, 2026).

1.9.4 Five common Hopper decode anti-patterns

1. **TMA everywhere** on $M{=}1$ micro-tiles. *Symptom:* TMA/MMA busy ratio <0.3 with flat ms/token (SemiAnalysis, 2024). *Debug:* compare vector-load path; reject tiles below $\sim 128{\times}128$ (Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

2. **TC utilization chasing** while bytes/token flat. *Symptom:* TC utilization rises, Table 1.6 ms/token unchanged (Chen, 2026). *Debug:* Nsight DRAM bytes first; return to pyramid level 1 (Davies *et al.*, 2025).

3. **Occupancy tuning** that accepts register spill. *Symptom:* high occupancy, rising `local memory loads` (Chen and YiRage Contributors, 2026). *Debug:* spill counters versus Table 1.7 high-water (Chen, 2026).

4. **Graph capture without fusion.** *Symptom:* launches/token collapse, bytes/token flat (Table 1.8, row 2) (NVIDIA, 2024). *Debug:* ablation row must show bytes moving, not only launches (Chen, 2026).

5. **Fusion without capacity sheet.** *Symptom:* kernel runs, shared high-water exceeds 227 KB usable, silent spill (NVIDIA, 2022). *Debug:* static Table 1.7; treat spill as compile failure (Chen and YiRage Contributors, 2026; Chen, 2026).

Table 1.9: Nsight Compute priorities for Hopper decode MegaKernels.

Counter / metric	**Interpretation**
DRAM throughput	Activation + KV traffic
Local memory loads	Spill—fusion bug
TMA busy / MMA busy	Pipeline balance
TC utilization	Tune only after bytes clean

1.9.5 Nsight counter cheat sheet

Prioritize: DRAM bytes per kernel → spill/local loads → TMA vs MMA busy → TC utilization last (Davies *et al.*, 2025; SemiAnalysis, 2024; Chen, 2026).

✓ **Practice checkpoint.** Attach Table 1.6-style before/after and shared-memory sheet to every Hopper fusion PR (Vellaisamy *et al.*, 2026; Chen, 2026).

Efficiency metric (optional fleet view). Track measured tokens/s divided by roofline tokens/s after subtracting framework scheduler overhead (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026). CUDA backend teams own fused-kernel numerator; framework teams own scheduler tax—publish both (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023).

1.10 Hopper Benchmarking and Verification

Chapter 1–2 define decode-cell metrics; this section adds *Hopper-specific* counters only (Vellaisamy *et al.*, 2026; Chen, 2026). Full CI checklists live in Appendix B (§**??**)—not repeated here (Chen and YiRage Contributors, 2026).

Four verification classes. **(1) Core metrics:** launches/token, bytes/token, ms/token at ctx $\in \{512, 2048, 8192\}$, BS=1 (Vellaisamy *et al.*, 2026; Chen, 2026). **(2) Environment:** pinned model weights, driver/CUDA versions, warmup including graph capture (NVIDIA, 2024; Davies *et al.*, 2025). **(3) Hopper extensions:** TMA transaction count, shared-memory high-water, graph bucket hit rate (SemiAnalysis, 2024; NVIDIA, 2022). **(4) CI gate:** ablation matrix (graph / fusion / both) + triplet legality summary (Chen and YiRage Contributors, 2026; AMD, 2024b).

Table 1.10: Hopper decode counter worksheet (extends Chapter 1).

Metric	**Hopper-specific meaning**
Launches/token	Graph replay vs eager
Bytes/token	Activation + KV + spill
TMA edges/token	Async copy orchestration
Shared high-water	Fusion budget violations
TC utilization	Often misleading at batch-1

Paged KV stress. Benchmark with vLLM block tables when product ships paging—TMA legality at 32K may differ from 2K (Kwon *et al.*, 2023; Dao *et al.*, 2023).

Warm-up protocol. Include graph capture, compile amortization, and $\geq$1000 steady decode steps before recording medians (NVIDIA, 2024; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Report first-token and steady ms/token separately—product SLOs may bind to either (Kwon *et al.*, 2023; Chen, 2026).

Appendix pointer. Fine-grained CI gates (driver pins, speculative decode sweeps, TP=4 benchmarks) are listed in Appendix B (§**??**) (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Liu *et al.*, 2024). The body of this chapter states principles; the appendix holds checkboxes (Vellaisamy *et al.*, 2026; Chen, 2026).

Triplet runs. Export the same decoder IR to CUDA and XDNA; diff legality before celebrating Hopper ms/token (Chen and YiRage Contributors, 2026; AMD, 2024b,a; Chen, 2020; Davies *et al.*, 2025). A CUDA-only win that breaks NPU compile should fail CI, not ship to heterogeneous fleets (AMD, 2024b; Chen and YiRage Contributors, 2026).

Closing verification sentence. Hopper benchmarks are complete when launches/token, bytes/token, and ms/token move together within tolerances on all standard ctx buckets—otherwise the change is a lab draft, not a fleet candidate (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

Example gate (one line). Llama-3.2-1B at ctx=2048 must show bytes/token within 10% of worksheet *and* launches/token within 5% of TaxBreak baseline;

attach Nsight DRAM delta (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). Full checkbox lists live in Appendix B (§**??**) (Chen and YiRage Contributors, 2026; Chen, 2020).

Observability and fleet pins. Instrument kernels with stable names; pin driver and CUDA in decode pools; re-run goodput cells after upgrades (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024; Davies *et al.*, 2025). Hopper-tuned artifacts are not portable across SKUs without re-roofline (Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024).

Multi-GPU note (one paragraph). Tensor-parallel decode splits heads across devices; allreduce placement changes launches/token even when single-GPU IR looks optimal (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). This chapter stays single-GPU batch-1; fleet TP sweeps belong in appendix checklists (Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023).

✓ **Practice checkpoint.** Promotion requires all three ctx buckets green on bytes/token *and* launches/token within published tolerances (Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026).

What we deliberately omit here. Nightly driver pins, speculative-decode draft graphs, multi-GPU tensor-parallel sweeps, and fleet OPEX models belong in appendix checklists and Chapter 24—not in the core Hopper narrative (Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024). This section states the verification *classes* engineers must satisfy; the appendix lists every checkbox (Chen and YiRage Contributors, 2026).

Ablation template. Every Hopper promotion ships three rows: graph-only, fusion-only, graph+fusion (Chen, 2026; NVIDIA, 2024; Vellaisamy *et al.*, 2026). Reviewers reject ms/token deltas that do not label which lever moved (Davies *et al.*, 2025).

Regression hygiene. Pin tokenizer, weights, ctx bucket, driver, CUDA, and YiRage commit for every comparison (Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026). Record median and p99 ms/token; launches and bytes must move with latency (Davies *et al.*, 2025; Kwon *et al.*, 2023).

Security of performance claims. Never cite peak TFLOPs for decode chapters—cite named models, ctx buckets, and all three goodput counters (Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016; Davies *et al.*, 2025).

Fleet configuration guidance. Pin driver, CUDA, and YiRage commit in decode pools; re-run launches/token and bytes/token after upgrades (Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; NVIDIA, 2024). Hopper-tuned kernels are not portable to L4 cells without re-roofline analysis (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Practitioner mantra. Profile end-to-end decode first; cut activation bytes second; collapse launches third; tune tensor cores fourth; re-profile on every driver and ctx bucket change (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; SemiAnalysis, 2024). Teach teammates to read residency diagrams before PTX diffs (Chen and YiRage Contributors, 2026; Chen, 2020).

Choosing the comparison window. Benchmarks that compare only a single ctx bucket hide fusion regressions at the longest decode step (Chen, 2026; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). When ablation rows disagree across buckets, trust the bucket your SLO actually binds and ship the row matrix, not one headline median (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

1.11 Key Takeaways

- **Priority principle.** Hopper decode: cut HBM activation traffic, then launches, then latency, then TC (Chen, 2026; Vellaisamy *et al.*, 2026). **Action:** follow Figure 1.1; reject PRs that invert the order (Davies *et al.*, 2025).
- **Residency boundary principle.** 227 KB usable shared memory and the register file are hard fusion caps—spill means failure (NVIDIA, 2022; Chen and YiRage Contributors, 2026). **Action:** attach static byte sheets; treat spill counters as build breaks (Chen, 2026).
- **Async copy principle.** TMA hides latency; fusion removes bytes—compose both, confuse neither (SemiAnalysis, 2024; Chen, 2026). **Action:** verify bytes/token dropped before celebrating TMA overlap (Davies *et al.*, 2025).
- **Minimal sync principle.** Warp shuffle → CTA sync → cluster barrier; never grid sync per token (Dao *et al.*, 2022; NVIDIA, 2022). **Action:** audit

barrier placement on fused DAG edges only (Chen and YiRage Contributors, 2026).

- **Verification principle.** launches/token, bytes/token, and ms/token must move together—TC utilization alone is invalid evidence (Vellaisamy *et al.*, 2026; Chen, 2026). **Action:** store Table 1.6 and Table 1.8 rows in CI artifacts; compare against Table 1.1 before merge (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

1.12 Conclusion

Hopper supplies TMA, clusters, tensor cores, and SFUs as mechanical-sympathy tools when schedules respect Table 1.2 (NVIDIA, 2022; SemiAnalysis, 2024; Dao *et al.*, 2022). This chapter gave an optimization path—not a feature catalog—from fusion through graphs, TMA, clusters, sync, and finally PTX/TC tuning (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026). Revisit Table 1.1 before every Hopper PR: fusion and graphs move goodput; TC polish is last (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026).

Chapter **??** contrasts dynamic CUDA tiers with AMD XDNA static tile SRAM: same bufferization graph, different legality rules (AMD, 2024b,a; Davies *et al.*, 2025). Carry your Hopper worksheet (launches/token, bytes/token, shared-memory high-water) into the triplet comparison—faster silicon does not redefine goodput (Vellaisamy *et al.*, 2026; Chen, 2026).

Worksheet gate. Before opening XDNA chapters, confirm Table 1.6 is filled for your model and pyramid level 1–2 are exhausted (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026). If bytes/token is still flat on Hopper, Chapter 5 will not rescue the schedule (AMD, 2024b,a).

Closing reminder. Hopper features reward schedules that respect Table 1.2; profilers should confirm wins, not discover basic spill (Chen, 2026; Davies *et al.*, 2025; NVIDIA, 2022; Chen and YiRage Contributors, 2026). Keep the Hopper worksheet in version control beside compiler config (Chen, 2020; Vellaisamy *et al.*, 2026).

Part II bridge. With Hopper bounds established, Chapter **??** shows how the same IR meets static XDNA tiles—the contrast clarifies why triplet bufferization

matters (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2020).

Done criteria. You are done when you can name your pyramid level, attach Table 1.6 and Table 1.8 rows, and explain which goodput counter proves the PR (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). Proceed to Chapter **??** with the Hopper worksheet in hand (AMD, 2024b,a; Chen, 2020).

Bibliography

AMD (2024a). Ai engine technology. https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html. 6, 12, 17, 27, 30, 31

AMD (2024b). AMD XDNA architecture. https://www.AMD.com/en/technologies/XDNA.html. 6, 8, 9, 12, 16, 17, 18, 21, 22, 23, 26, 27, 30, 31

Chen, J. (2026). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31

Chen, X. (2020). Compiler-based hardware profiling for software-defined architecture. 1, 2, 4, 6, 7, 8, 10, 11, 12, 13, 15, 16, 17, 18, 20, 21, 22, 23, 24, 27, 28, 29, 30, 31

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. https://github.com/chenxingqiang/YiRage. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 1, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31

Dao, T. *et al.* (2023). Flash-decoding for long-context inference. PyTorch blog and Stanford CRFM technical report. Parallelizes attention along KV sequence; up to 8× decode speedup at long context. 3, 4, 8, 9, 10, 11, 12, 13, 14, 15, 16, 18, 20, 22, 24, 27

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 19, 20, 21, 22, 23, 24, 29, 30

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more decode than prefill nodes. 2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 13, 14, 15, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 3, 4, 5, 6, 8, 9, 10, 12, 13, 16, 19, 20, 21, 22, 25, 29

IREE / iree-org contributors (2024). IREE: Intermediate representation execution environment — an MLIR-based end-to-end compiler and runtime. `https://iree.dev`. AOT MLIR compiler+runtime; Flow/Stream/HAL/VM dialects; dispatch-region fusion; `.vmfb` fat binary; targets llvm-cpu, CUDA, ROCm/HIP, Vulkan/SPIR-V, Metal, AMD AIE (experimental). 17, 18

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. 5, 6, 7, 8, 9, 10, 18, 20, 21, 22, 24, 26, 27, 28, 29

Liu, A. *et al.* (2024). Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*. 3, 6, 14, 15, 16, 17, 18, 20, 21, 22, 27, 28

NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 17, 18, 19, 20, 21, 23, 25, 26, 29, 30

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 2, 4, 10, 11, 12, 13, 18, 19, 21, 22, 23, 24, 25, 26, 27, 28, 29

Rotem, N., Fix, J., Abdulrasool, S., Catron, G., Deng, S., Dzhabarov, R., Gibson, N., Hegeman, J., Lele, M., Levenstein, R., Montgomery, J., Maher, B., Nadathur, S., Olesen, J., Park, J., Rakhov, A., and Smelyanskiy, M. (2018). Glow: Graph lowering compiler techniques for neural networks. *arXiv preprint arXiv:1805.00907*. Two-phase strongly-typed IR (high-level graph + low-level address-only); lowering to linear-algebra primitives; profile-guided quantization; AOT bundles. 8

SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter. 1, 2, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 19, 20, 21, 23, 24, 25, 26, 29, 30

Shah, J., Bikshandi, G., Zhang, Y., Thakkar, V., Ramani, P., and Dao, T. (2024). FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. In *Advances in Neural Information Processing Systems (NeurIPS)*. Hopper warp specialization (producer TMA / consumer WGMMA), persistent kernels with tile scheduler, pingpong scheduling. 7, 10, 15, 19

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31

Index

Hopper, 4

Tensor core, 4
TMA, 4